Introduction:

The goal of this Cooking Class Platform is to design and develop a relational database for a Cooking Class Platform. This architecture allows the system to manage links and relationships between students, teachers, cooking classes, scheduled sessions, and enrollments. The database was designed to minimize unnecessary data and preserve referential integrity while accurately representing the relationships among these elements.

Requestment Gathering:

https://weezevent.com/en-gb/blog/organise-cooking-classes/

This system would save and manage information about specific cooking classes, in addition to maintaining student information, instructor information, and teaching specialties, according to the article on how to set up a cooking class. The Cooking Class Platform's design, requirements, and potential connections are as follows:

Requirement	**Design decision**
Classes can run more than once	Separate *CookingClass* and *ClassSession*
Students can attend many Class Sessions.	Establish *Student* - M: N - *ClassSession* Relationship
Sessions contains many students	Resolve through *Enrollment*
Classes need instructors	*Instructor* - 1:N - *CookingClass*
Registration needs status	Add *EnrollmentStatus*

Business Rules:

The following are the business principles that will govern how a Cooking Class Platform micro world operates. The first rule states that a single teacher may teach several cooking classes, but each cooking class is allocated to a single instructor. The second business guideline

states that even though a cooking class may have many scheduled sessions, each class session is a part of a single cooking class. One student may enroll in multiple class sessions, and one class session may have numerous students. This is the next business guideline that must be adhered to for this mini-world. The third criterion is that a student may enroll just once in a given class session and that each cooking lesson has a standard price, cuisine type, and difficulty level. Another guideline the Cooking class platform functions on is that each class session includes planned date/time, duration, capacity, location, and status parameters for enrollers to be aware of. Last but not least, a business rule states that every enrollment row of data contains the student's enrollment date and current status.

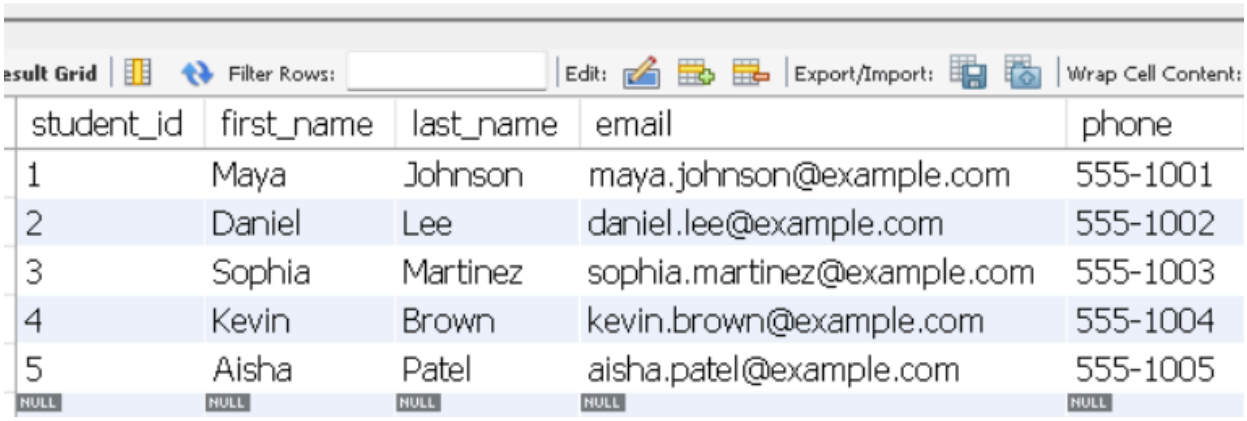
Conceptual Design:

Here are the entities/ tables that would be used in constructing the database for the Cooking Class Platform:

Entity	Purpose
Student	Accounts for people who will be taking cooking classes.
Instructor	Accounts for people who will be teaching the cooking classes.
CookingClass	Description of the cooking classes, such as the class name, pricing for the class, and so on.
ClassSession	A scheduled occurrence of the available cooking classes.
Enrollment	Junction table which connects *Student* and *ClassSession* tables.

STUDENT TABLE:

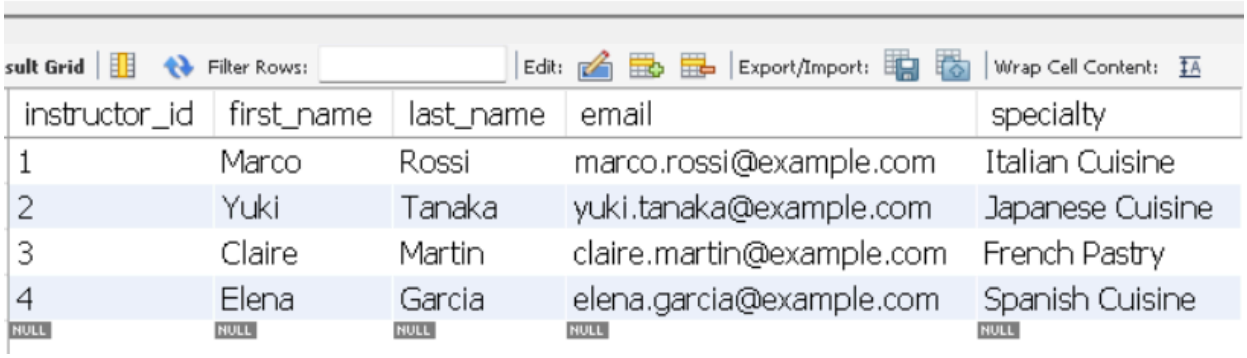
```
90 • SELECT * FROM student;
91
```



student_id	first_name	last_name	email	phone
1	Maya	Johnson	maya.johnson@example.com	555-1001
2	Daniel	Lee	daniel.lee@example.com	555-1002
3	Sophia	Martinez	sophia.martinez@example.com	555-1003
4	Kevin	Brown	kevin.brown@example.com	555-1004
5	Aisha	Patel	aisha.patel@example.com	555-1005
NULL	NULL	NULL	NULL	NULL

INSTRUCTOR TABLE:

```
102 • SELECT * FROM instructor;
103
```



instructor_id	first_name	last_name	email	specialty
1	Marco	Rossi	marco.rossi@example.com	Italian Cuisine
2	Yuki	Tanaka	yuki.tanaka@example.com	Japanese Cuisine
3	Claire	Martin	claire.martin@example.com	French Pastry
4	Elena	Garcia	elena.garcia@example.com	Spanish Cuisine
NULL	NULL	NULL	NULL	NULL

CookingClass TABLE:

```
115 • SELECT * FROM cooking_class ;
116
```

sult Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

class_id	instructor_id	class_name	cuisine_type	difficulty_level	standard_price
1	1	Italian Pasta Basics	Italian	Beginner	65.00
2	2	Sushi Workshop	Japanese	Intermediate	80.00
3	3	French Pastry Basics	French	Beginner	70.00
4	4	Spanish Tapas Night	Spanish	Beginner	75.00
5	1	Advanced Italian Sauces	Italian	Advanced	90.00
NULL	NULL	NULL	NULL	NULL	NULL

ClassSession Table:

```
129 • SELECT * FROM class_session;
130
```

sult Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

session_id	class_id	start_datetime	duration_minutes	capacity	location	status
1	1	2026-10-03 18:00:00	120	16	Kitchen A	Scheduled
2	1	2026-10-10 18:00:00	120	16	Kitchen A	Scheduled
3	2	2026-10-05 17:30:00	150	12	Kitchen B	Scheduled
4	3	2026-10-08 18:30:00	120	14	Pastry Lab	Scheduled
5	4	2026-10-12 18:00:00	120	16	Kitchen C	Scheduled
6	5	2026-10-17 17:00:00	150	12	Kitchen A	Scheduled
NULL	NULL	NULL	NULL	NULL	NULL	NULL

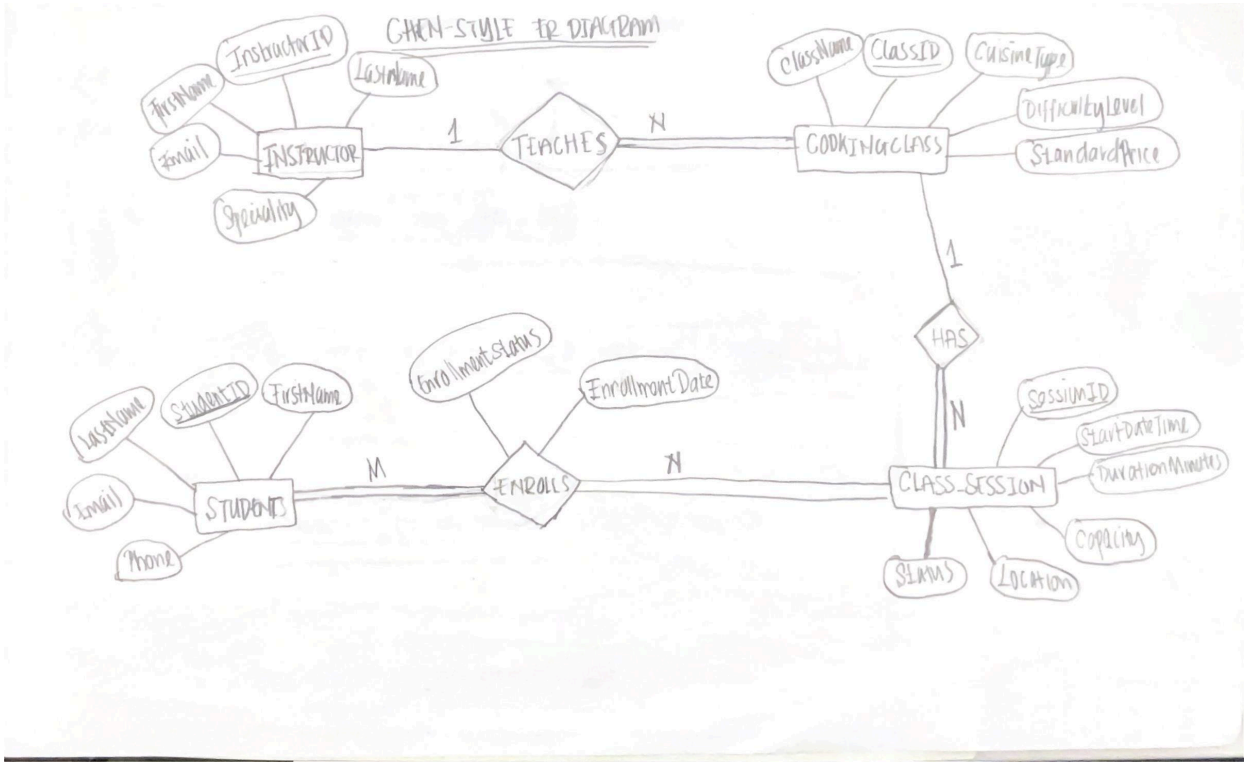
Enrollment Table:

```
151 • SELECT * FROM enrollment;
```

esult Grid | Filter Rows: | Edit: | Export/Import: | Wrap

student_id	session_id	enrollment_date	enrollment_status
1	1	2026-09-25 10:00:00	Registered
1	3	2026-09-26 11:00:00	Registered
2	1	2026-09-25 10:15:00	Registered
2	4	2026-09-27 14:00:00	Registered
3	1	2026-09-26 09:30:00	Registered
3	5	2026-09-28 10:30:00	Registered
4	3	2026-09-26 12:00:00	Registered
4	5	2026-09-28 11:30:00	Registered
5	4	2026-09-27 15:00:00	Registered
5	6	2026-09-28 16:00:00	Registered

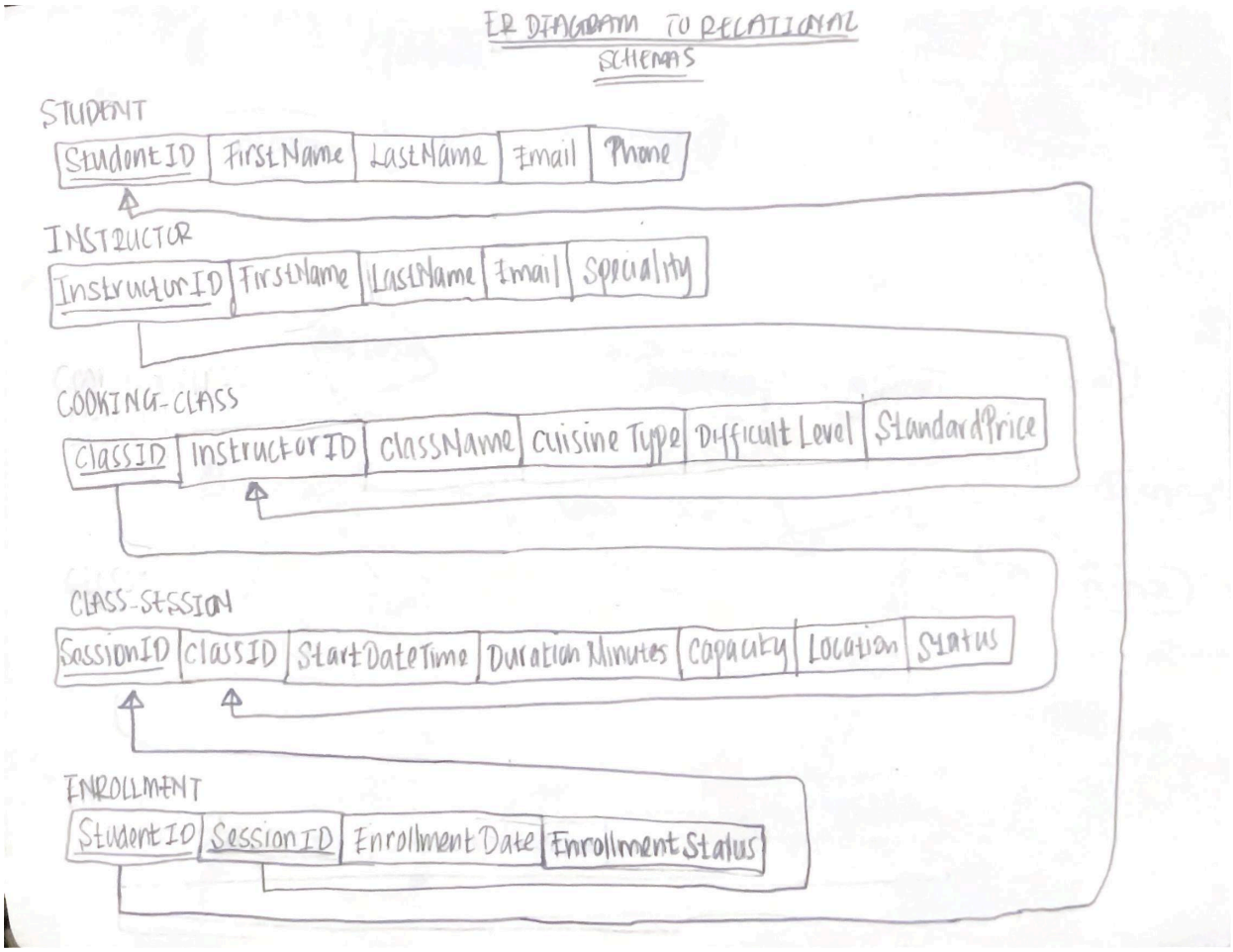
Chen ER-Diagram:



ER-to-Relational Mapping:

The relationships that the database works with are displayed in this relational mapping. It starts with the first relationship: each cooking class instructor will be able to teach multiple cooking classes, which will aid in the formation of a 1:N relationship. This signifies that there will be a foreign foreign foreign foreign foreign foreign foreign class class class class class entity. Then, the next relationship established in this database is that each CookingClass available has many ClassSessions to register for, which establishes a 1:N relationship, which means that the primary key attributes (underlined) from the CookingClass entity will be included as a foreign key attribute in the ClassSession relation on the N-side. The final and most significant relationship is the M: N relationship between the ClassSession and Student entities. The Enrollment table has the attributes Enrollment Status and Enrollment Date, and the

relationship means that the primary keys of the Student and ClassSession entities serve as both primary and foreign keys in the Enrollment table.



Challenges and Design Decisions:

Determining whether a cooking class and a planned session should be represented as a single entity was one of the key design choices. The culinary class may be offered. They may be offered. The interaction between students and class sessions was another crucial choice. While there might be confusion about the need for two tables for the cooking classes being offered, the reason behind the choice to identify the tables separately is that the Cooking Class table has

information about the desired cooking class that will be conducted will be about with info like the pricing of the class, the cuisine that will be taught by the instructor for the class, and so on. Then, for the Class_Session table, which serves as a connection to the student who would like to enroll in the available classes, with information like the time the class will take place, the capacity (how many people it will accommodate, the location of the class, and so on. The relationship is many-to-many and is handled by the enrollment table since a student may attend multiple sessions and a session may have numerous students. The Enrollment database additionally keeps information relevant to the registration itself, such as enrollment date and status.

Outcome:

The completed database properly represents the key functions of a Cooking Class Platform. While enforcing relationships through primary and foreign keys, it maintains structured records for students, instructors, classes, scheduled sessions, and enrollments for individuals interested in taking cooking classes. The primary relationships are one-to-many and many-to-many, according to the business rules between two tables that I studied during the implementation of the Cooking Class Platform database. SQL queries can be used to obtain helpful organizational data, such as forthcoming sessions, student registrations, instructor assignments, and enrollment totals, once the database has been implemented and data has been inserted into each table.